



Celebal Summer Internship 2026

Week 2 Task: E-Commerce Sales Database

SECTION – A

Q1. Write a query to display all columns and rows from the customer's table.

```
125
126 -- Q1. Write a query to display all columns and rows from the customer's table. --
127 • Select * From customers;
128
```

Result Grid								
Filter Rows:								
Edit: Export/Import: Wrap Cell Content:								
	customer_id	first_name	last_name	email	city	state	join_date	is_premium
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q2. Retrieve only the first_name, last_name, and city of all customers.

```
128 -- Q2. Retrieve only the first_name, last_name, and city of all customers. --
129 • select
130     first_name,
131     last_name,
132     city
133 From customers;
134
```

Result Grid			
Filter Rows:			
Export: Wrap Cell Content:			
	first_name	last_name	city
▶	Aarav	Sharma	Mumbai
	Priya	Patel	Ahmedabad
	Rohan	Gupta	Delhi
	Sneha	Reddy	Hyderabad
	Vikram	Singh	Jaipur
	Ananya	Iyer	Chennai
	Karan	Mehta	Pune
	Divya	Nair	Kochi

Q3. List all unique categories available in the products table.

```
135
136 -- Q3. List all unique categories available in the products table. --
137 • Select Distinct
138     category
139 from products;
140
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	category
▶	Clothing
	Electronics
	Home

Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL.

1. For table = Customers.

```
141 -- Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL. --
142 • show tables;
143 • describe customers;
144 IN this customer_id = PRI
145
146 describe orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Field	Type	Null	Key	Default	Extra
▶	customer_id	int	NO	PRI	NULL	
	first_name	varchar(50)	NO		NULL	
	last_name	varchar(50)	NO		NULL	
	email	varchar(100)	NO	UNI	NULL	
	city	varchar(50)	NO	MUL	NULL	
	state	varchar(50)	NO	MUL	NULL	
	join_date	date	NO		NULL	
	is_premium	tinyint(1)	YES		0	

2.For table = orders.

```
145
146 describe orders;
147 IN this order_id = PRI
148
149 describe products;
150 IN this product_id = PRI
151
152 describe order items;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	Field	Type	Null	Key	Default	Extra
▶	order_id	int	NO	PRI	NULL	
	customer_id	int	NO	MUL	NULL	
	order_date	date	NO	MUL	NULL	
	status	varchar(20)	NO	MUL	Pending	
	total_amount	decimal(12,2)	NO		NULL	

3.For table = products.

```
148
149 describe products;
150 IN this product_id = PRI
151
152 describe order items;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	Field	Type	Null	Key	Default	Extra
▶	product_id	int	NO	PRI	NULL	
	product_name	varchar(100)	NO		NULL	
	category	varchar(50)	NO	MUL	NULL	
	brand	varchar(50)	NO		NULL	
	unit_price	decimal(10,2)	NO		NULL	
	stock_qty	int	NO		0	

4.For table = order_items.

```
151
152 describe order_items;
153 IN this item_id = PRI
154
155 Primary key must be unique to ensure that each row has is identifiabe separately and each row has a seprate and unique identity.it ensures that
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Field	Type	Null	Key	Default	Extra
▶	item_id	int	NO	PRI	NULL	
	order_id	int	NO	MUL	NULL	
	product_id	int	NO	MUL	NULL	
	quantity	int	NO		NULL	
	unit_price	decimal(10,2)	NO		NULL	
	discount_pct	decimal(5,2)	YES		0.00	

Q5. What constraints are applied to the email column in the customers table? What would happen if you tried to insert a duplicate email?

Limit to 1000 rows

```

160 -- Q5. What constraints are applied to the email column in the customers table? What would happen if you tried to insert a duplicate email? --
161 UNIQUE constraint is applied to the email column to ensure that the no repeated data or email address must be there or to avoid redundancy.
162 NOT NULL is also used sometimes to ensure that every email has a email value.
163 SO, we use unique and not null constraints to avoid duplicate values and to ensure that every row has a value .
164 IF we tried to enter it will instantly give a error:"Error : Duplicate entry "
165 ex-
166 create table Tech(
167     email varchar(78) unique not null);
168 • insert into tech values ('harshita638@gmail.com'),('huwyddd928@gmail.com'),('harshita638@gmail.com');
169
170 ✖ So in above example i tried to enter a duplicate value and i receive a error i.e:"Error Code: 1062. Duplicate entry 'harshita638@gmail.com' for
171

```

Output

#	Time	Action	Message
✖ 1	13:04:00	Select * From customers LIMIT 0, 1000	Error Code: 1046. No database selected Select the default DB to be used by double-clicki
✖ 2	13:04:07	Create database Celebal	Error Code: 1007. Can't create database 'celebal'; database exists
✓ 3	13:04:14	Use Celebal	0 row(s) affected
✓ 4	13:04:21	Select * from orders LIMIT 0, 1000	13 row(s) returned
✓ 5	13:04:30	select first_name, last_name, city From customers LIMIT 0, 1000	8 row(s) returned
✓ 6	13:04:35	Select Distinct category from products LIMIT 0, 1000	3 row(s) returned
✓ 7	13:05:30	Select Distinct category from products LIMIT 0, 1000	3 row(s) returned
✓ 8	13:07:47	describe customers	8 row(s) returned
✓ 9	13:08:43	describe orders	5 row(s) returned
✓ 10	13:09:29	describe products	6 row(s) returned
✓ 11	13:10:09	describe order_items	6 row(s) returned
✖ 12	13:11:05	create table Tech(email varchar(78) unique not null)	Error Code: 1050. Table 'tech' already exists
✖ 13	13:11:13	create table Tech(email varchar(78) unique not null)	Error Code: 1050. Table 'tech' already exists

Q6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it? Write both the INSERT statement and explain the error.

Limit to 1000 rows

```

172 -- Q6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it? Write both the INSERT statement and explai
173 WE cant insert -50 as a unit_price here, if we tried to do so it will return an error .
174 Check constarint willll prevent it.
175 As in products table we applied
176 check (unit_price > 0),
177 so it will check the condition if condition is not get fulfilled it will not allow us to insert that value and immediatly return an error.
178 ex=
179 for unit_price=-50:-
180 INSERT INTO products
181 (product_id, product_name, category, brand, unit_price, stock_qty)
182 VALUES
183 (101, 'Laptop', 'Electronics', 'Dell', -50, 10);

```

Automatic context help is disabled. Use the toolbar manually get help for the current caret position or toggle automatic help.

Context Help Snippets

Output

#	Time	Action	Message	Duration / Fetch
✖ 1	13:04:00	Select * From customers LIMIT 0, 1000	Error Code: 1046. No database selected Select the default DB to be used by double-clicking its name in the SCH...	0.000 sec
✖ 2	13:04:07	Create database Celebal	Error Code: 1007. Can't create database 'celebal'; database exists	0.000 sec
✓ 3	13:04:14	Use Celebal	0 row(s) affected	0.000 sec
✓ 4	13:04:21	Select * from orders LIMIT 0, 1000	13 row(s) returned	0.000 sec / 0.000 sec
✓ 5	13:04:30	select first_name, last_name, city From customers LIMIT 0, 1000	8 row(s) returned	0.016 sec / 0.000 sec
✓ 6	13:04:35	Select Distinct category from products LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
✓ 7	13:05:30	Select Distinct category from products LIMIT 0, 1000	3 row(s) returned	0.032 sec / 0.000 sec
✓ 8	13:07:47	describe customers	8 row(s) returned	0.000 sec / 0.000 sec
✓ 9	13:08:43	describe orders	5 row(s) returned	0.016 sec / 0.000 sec
✓ 10	13:09:29	describe products	6 row(s) returned	0.000 sec / 0.000 sec
✓ 11	13:10:09	describe order_items	6 row(s) returned	0.031 sec / 0.000 sec
✖ 12	13:11:05	create table Tech(email varchar(78) unique not null)	Error Code: 1050. Table 'tech' already exists	0.000 sec
✖ 13	13:11:13	create table Tech(email varchar(78) unique not null)	Error Code: 1050. Table 'tech' already exists	0.000 sec
✖ 14	13:12:32	for unit_price=-50:- INSERT INTO products (product_id, product_name, category, brand, unit_price, stock_qt...	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL serv...	0.000 sec

SECTION – B

Q7. Retrieve all orders with status = 'Delivered'.

The screenshot shows a SQL IDE interface. The top part is a query editor with the following SQL code:

```
-- Section 8 - Filtering & Optimization (WHERE, Indexes) --
-- Q7. Retrieve all orders with status = 'Delivered'. --
select * from orders
where status = 'Delivered';
-- Q8. Find all products in the 'Electronics' category with a unit price greater than $2000 --
```

The bottom part of the screenshot shows a results grid displaying the output of the first query. The grid has a table with the following columns: order_id, customer_id, order_date, status, and total_amount. The table contains 10 rows of data, all with a status of 'Delivered'.

order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-01	Delivered	2000.00
1002	102	2024-08-03	Delivered	799.00
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1008	103	2024-08-20	Delivered	899.00
1010	108	2024-08-28	Delivered	1598.00
NULL	NULL	NULL	NULL	NULL

Q8. Find all products in the 'Electronics' category with a unit price greater than ₹2000.

```
195 -- Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000. --
196 • Select * FROM products
197 Where category = 'Electronics' AND unit_price > 2000;
198
```

	product_id	product_name	category	brand	unit_price	stock_qty
▶	203	Smart Watch	Electronics	Noise	2999.00	150
•	205	Bluetooth Speaker	Electronics	JBL	3499.00	200

products 11 ×

Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.

[illegible]

Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.

```
202
203  -- Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled. --
204  Select * from orders
205  Where status != 'Cancelled' And order_date between '2024-08-10' AND '2024-08-25';
```

Result Grid

	order_id	customer_id	order_date	status	total_amount
▶	1004	101	2024-08-10	Delivered	3499.00
	1006	105	2024-08-15	Delivered	5898.00
	1007	106	2024-08-18	Pending	1299.00
	1008	103	2024-08-20	Delivered	899.00
	1009	107	2024-08-25	Shipped	6098.00
•	NULL	NULL	NULL	NULL	NULL

Q11. Explain what the index idx_orders_date does. How would it improve the performance of a query that filters orders by order_date? Write a sample query that would benefit from this index.

```
207  -- Q11. Explain what the index idx_orders_date does. How would it improve the performance of a query that filters orders by order_date? Write a
208  The idx_orders_date helps find the order date fast and easily . it is created on the order_date column of the order column.
209  Instead of scanning every row in the table, the database can quickly locate matching records using the index, making queries faster, especially
210  ex-
211  SELECT * FROM orders
212  WHERE order_date = '2024-08-15';
```

Result Grid

	order_id	customer_id	order_date	status	total_amount
▶	1006	105	2024-08-15	Delivered	5898.00
•	NULL	NULL	NULL	NULL	NULL

orders 15 x

Apply Revert

Output

Q12. If you run: `SELECT * FROM customers WHERE YEAR(join_date) = 2024;` — would the index on `join_date` be used? Explain why or why not, and rewrite the query to be index-friendly (SARGable).

`SELECT * FROM customers WHERE YEAR(join_date) = 2024;`

```

213
214 -- Q12. If you run: SELECT * FROM customers WHERE YEAR(join_date) = 2024; — would the index
215 • SELECT * FROM customers
216 WHERE YEAR(join_date) = 2024;
217 ✖ No the index on join_date will not be used. the database has to chk every row to find match.
218 this is because we used year function in this.

```

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Rewritten indexed friendly query.

```

213
214 -- Q12. If you run: SELECT * FROM customers WHERE YEAR(join_date) = 2024; — would the index on join_date be
215 • SELECT * FROM customers
216 WHERE YEAR(join_date) = 2024;
217 ✖ No the index on join_date will not be used. the database has to chk every row to find match.
218 this is because we used year function in this.
219 we have to make it index friendly to make it execute faster.
220 query-
221 Select * from orders
222 Where status != 'Cancelled' And order_date between '2024-08-10' AND '2024-08-25';
223 ✖ This query is fast because the database will directly check in the order date column and fetch the result.

```

order_id	customer_id	order_date	status	total_amount
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1007	106	2024-08-18	Pending	1299.00
1008	103	2024-08-20	Delivered	899.00
1009	107	2024-08-25	Shipped	6098.00
NULL	NULL	NULL	NULL	NULL

SECTION – C

Q13. Count the total number of orders in the orders table.

```
227
228 -- Q13. Count the total number of orders in the orders table. --
229 select
230     count(order_id) as total_orders
231 from orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	total_orders
▶	13

Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.

```
232
233 -- Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders. --
234 • select
235     status,
236     sum(total_amount) as revenue
237 from orders
238 where status = 'Delivered';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	status	revenue
▶	Delivered	14693.00

Q15. Calculate the average unit_price of products in each category.

Limit to 1000 rows

```
240 -- Q15. Calculate the average unit_price of products in each category. --
241 • select
242     category,
243     avg(unit_price) as avg_price
244 from products
245 GROUP BY Category;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	category	avg_price
▶	Clothing	2699.000000
	Electronics	2224.000000
	Home	949.000000

Q16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.

```

246
247 -- Q16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order. --
248 • select
249     status,
250     count(order_id) as total_orders,
251     sum(total_amount) as total_revenue
252 from orders
253 group by status
254 order by total_revenue desc;
255

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	status	total_orders	total_revenue
▶	Delivered	6	14693.00
	Shipped	2	13596.00
	Pending	4	5897.00
	Cancelled	1	2999.00

Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.

```

256 -- Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category. --
257 • SE Execute the selected portion of the script or everything, if there is no selection
258     category,
259     MAX(product_name) AS expensive_product,
260     MAX(unit_price) AS expensive_price,
261     MIN(product_name) AS cheapest_product,
262     MIN(unit_price) AS cheapest_price
263 FROM products
264 GROUP BY category;
265

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	category	expensive_product	expensive_price	cheapest_product	cheapest_price
▶	Clothing	Running Shoes	4599.00	Cotton T-Shirt	799.00
	Electronics	Wireless Earbuds	3499.00	Bluetooth Speaker	899.00
	Home	Cushion Covers (Set)	1299.00	Bedsheet Set	599.00

Q18. List all product categories where the average unit_price is greater than ₹2000. (Hint: Use HAVING clause)

```
265
266 -- Q18. List all product categories where the average unit_price is greater than ₹2000. (Hint: Use HAVING clause) --
267 • SELECT
268     category,
269     AVG(unit_price) AS avg_price
270 FROM products
271 GROUP BY category
272 HAVING AVG(unit_price) > 2000;
273
274 -- Section D - Joins & Relationships --
```

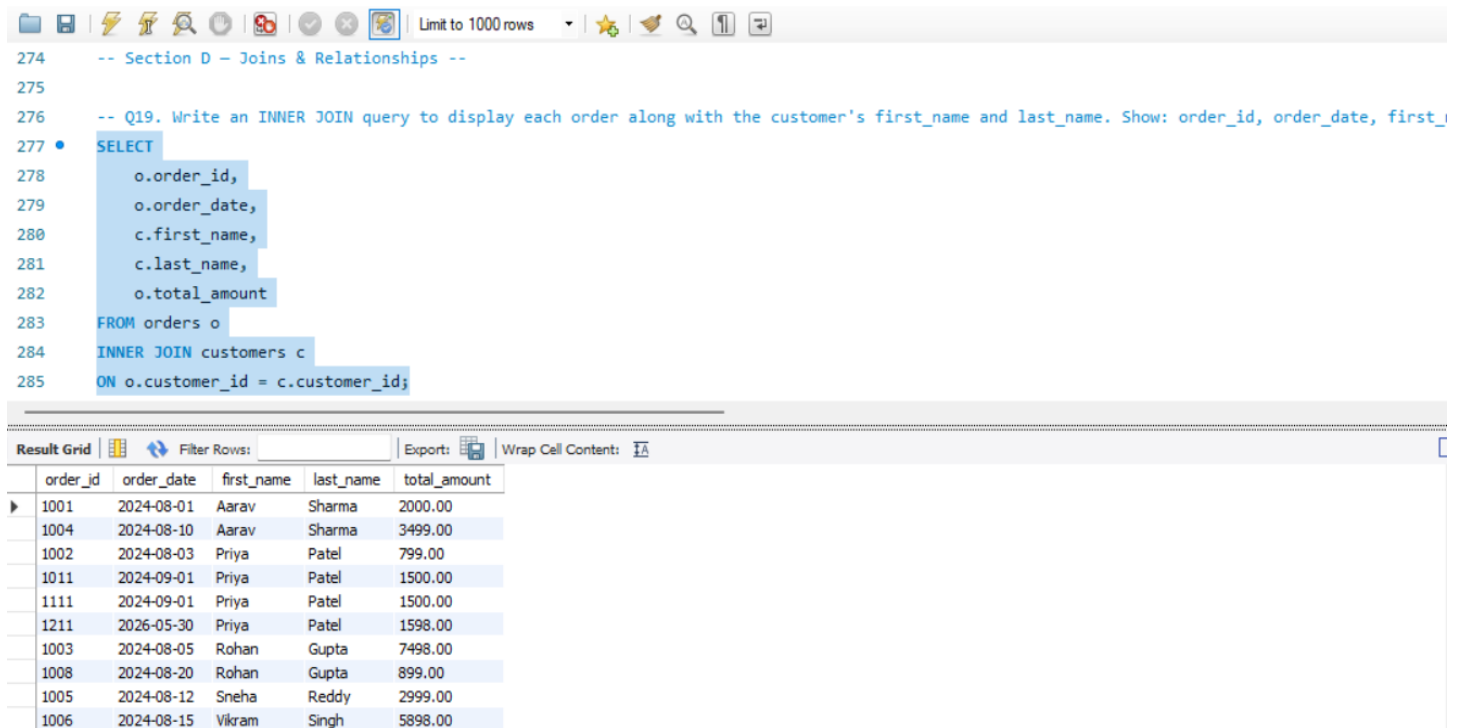
Result Grid   Filter Rows: Export:  Wrap Cell Content: 

	category	avg_price
▶	Clothing	2699.000000
	Electronics	2224.000000

SECTION – D

Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name.

Show: order_id, order_date, first_name, last_name, total_amount.



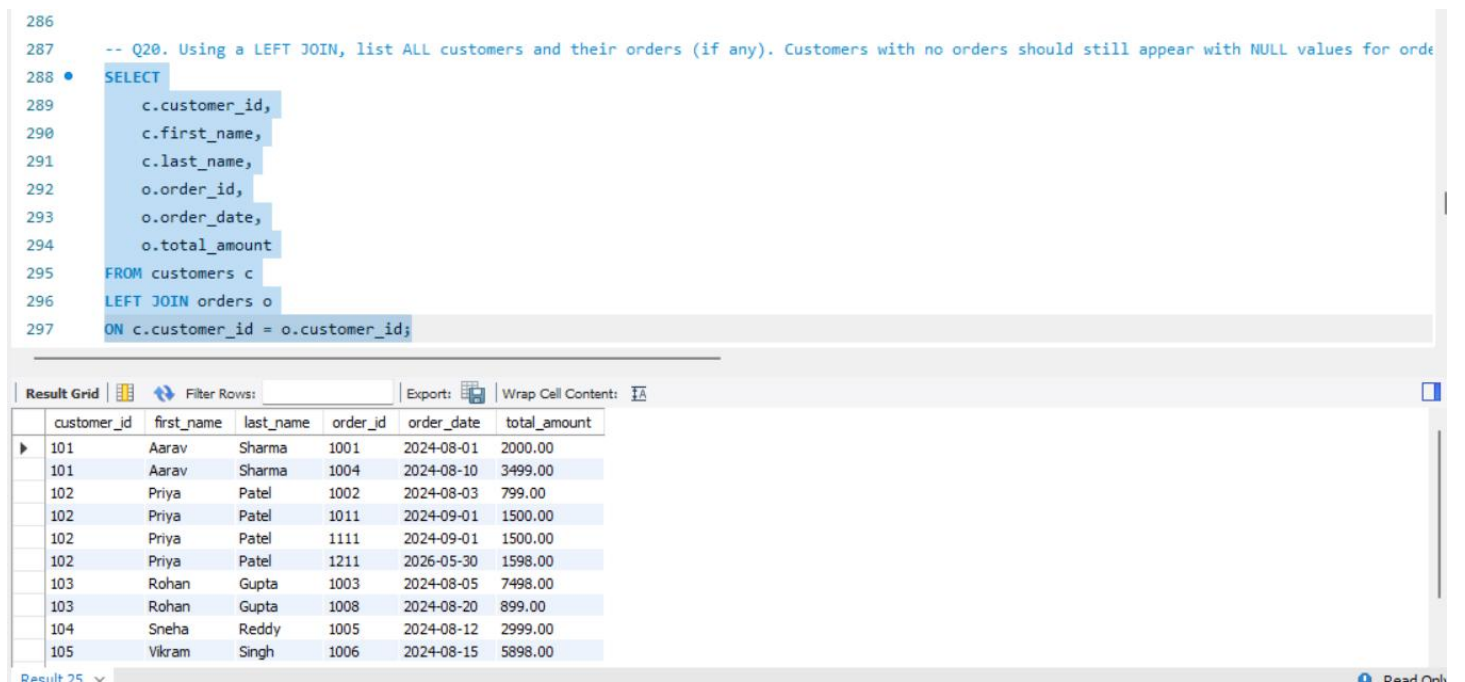
The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains the following code:

```
-- Section D – Joins & Relationships --  
  
-- Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name. Show: order_id, order_date, first_  
SELECT  
    o.order_id,  
    o.order_date,  
    c.first_name,  
    c.last_name,  
    o.total_amount  
FROM orders o  
INNER JOIN customers c  
ON o.customer_id = c.customer_id;
```

Below the editor is the 'Result Grid' showing the query results. The columns are order_id, order_date, first_name, last_name, and total_amount. The results are as follows:

order_id	order_date	first_name	last_name	total_amount
1001	2024-08-01	Aarav	Sharma	2000.00
1004	2024-08-10	Aarav	Sharma	3499.00
1002	2024-08-03	Priya	Patel	799.00
1011	2024-09-01	Priya	Patel	1500.00
1111	2024-09-01	Priya	Patel	1500.00
1211	2026-05-30	Priya	Patel	1598.00
1003	2024-08-05	Rohan	Gupta	7498.00
1008	2024-08-20	Rohan	Gupta	899.00
1005	2024-08-12	Sneha	Reddy	2999.00
1006	2024-08-15	Vikram	Singh	5898.00

Q20. Using a LEFT JOIN, list ALL customers and their orders (if any). Customers with no orders should still appear with NULL values for order columns.



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains the following code:

```
-- Q20. Using a LEFT JOIN, list ALL customers and their orders (if any). Customers with no orders should still appear with NULL values for orde  
SELECT  
    c.customer_id,  
    c.first_name,  
    c.last_name,  
    o.order_id,  
    o.order_date,  
    o.total_amount  
FROM customers c  
LEFT JOIN orders o  
ON c.customer_id = o.customer_id;
```

Below the editor is the 'Result Grid' showing the query results. The columns are customer_id, first_name, last_name, order_id, order_date, and total_amount. The results are as follows:

customer_id	first_name	last_name	order_id	order_date	total_amount
101	Aarav	Sharma	1001	2024-08-01	2000.00
101	Aarav	Sharma	1004	2024-08-10	3499.00
102	Priya	Patel	1002	2024-08-03	799.00
102	Priya	Patel	1011	2024-09-01	1500.00
102	Priya	Patel	1111	2024-09-01	1500.00
102	Priya	Patel	1211	2026-05-30	1598.00
103	Rohan	Gupta	1003	2024-08-05	7498.00
103	Rohan	Gupta	1008	2024-08-20	899.00
104	Sneha	Reddy	1005	2024-08-12	2999.00
105	Vikram	Singh	1006	2024-08-15	5898.00

Q21. Write a query using JOINS across three tables (orders → order_items → products) to show: order_id, product_name, quantity, unit_price, and discount_pct for each order item.

```

298
299 -- Q21. Write a query using JOINS across three tables (orders + order_items + products) to show: order_id, product_name, quantity, unit_price,
300 • SELECT
301     o.order_id,
302     p.product_name,
303     oi.quantity,
304     oi.unit_price,
305     oi.discount_pct
306 FROM orders o
307     INNER JOIN order_items oi
308     ON o.order_id = oi.order_id
309     INNER JOIN products p
310     ON oi.product_id = p.product_id;

```

order_id	product_name	quantity	unit_price	discount_pct
1001	Wireless Earbuds	2	1499.00	0.00
1006	Wireless Earbuds	1	1499.00	10.00
1002	Cotton T-Shirt	1	799.00	0.00
1003	Smart Watch	1	2999.00	0.00
1005	Smart Watch	1	2999.00	0.00
1003	Running Shoes	1	4599.00	5.00
1006	Running Shoes	1	4599.00	5.00
1004	Bluetooth Speaker	1	3499.00	0.00
1009	Bluetooth Speaker	1	3499.00	0.00

Q22. Explain the difference between LEFT JOIN and RIGHT JOIN with an example from this schema. When would you use a FULL OUTER JOIN?

Left join:-

```

311
312 -- Q22. Explain the difference between LEFT JOIN and RIGHT JOIN with an example from this schema. When would you use a FULL OUTER JOIN? --
313 • A 'LEFT JOIN' returns all records from the left table and matching records from the right table.
314 If there is no matching data in the right table, 'NULL' values are shown.
315
316 Example:
317 SELECT
318     c.first_name,
319     o.order_id
320 FROM customers c
321 LEFT JOIN orders o
322 ON c.customer_id = o.customer_id;
323
324 • This query shows all customers, even if they have no orders.
325
326 A 'RIGHT JOIN' returns all records from the right table and matching records from the left table.
327 If there is no matching data in the left table, 'NULL' values are shown.
328
329 Example:
330 SELECT
331     o.order_id,
332     c.first_name
333 FROM orders o
334 RIGHT JOIN customers c
335 ON o.customer_id = c.customer_id;

```

first_name	order_id
Aarav	1001
Aarav	1004
Priya	1002
Priya	1011
Priya	1111

Right join: -

```
325
326 A 'RIGHT JOIN' returns all records from the right table and matching records from the left table.
327 If there is no matching data in the left table, 'NULL' values are shown.
328
329 Example:
330 SELECT
331     c.first_name,
332     o.order_id
333 FROM customers c
334 RIGHT JOIN orders o
335 ON c.customer_id = o.customer_id;
```

336 This query shows all orders, even if customer details are missing.

337

338 A 'FULL OUTER JOIN' is used when we want all records from both tables, including matching and non-matching rows.

339 It is useful when we want complete data from both tables. MySQL does not directly support 'FULL OUTER JOIN'.

340

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	first_name	order_id
▶	Aarav	1001
	Aarav	1004
	Priya	1002
	Priya	1011
	Priya	1111

Result 28 x

Output

Q23. Identify all Foreign Key relationships in the schema. Explain what would happen if you tried to insert an order with customer_id = 999 (which doesn't exist in customers).

Foreign key relationship:-

```
340
341 -- Q23. Identify all Foreign Key relationships in the schema. Explain what would happen if you tried to insert an order with customer_id = 999
342 SELECT
343     TABLE_NAME AS child_table,
344     COLUMN_NAME AS foreign_key_column,
345     REFERENCED_TABLE_NAME AS parent_table,
346     REFERENCED_COLUMN_NAME AS referenced_column
347 FROM
348     INFORMATION_SCHEMA.KEY_COLUMN_USAGE
349 WHERE
350     REFERENCED_TABLE_NAME IS NOT NULL
351     AND TABLE_SCHEMA = DATABASE();
```

352

353 Foreign Key Relationships in the Schema:-

354

355 orders.customer_id → customers.customer_id

356 order_items.order_id → orders.order_id

357 order_items.product_id → products.product_id

358

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	child_table	foreign_key_column	parent_table	referenced_column
▶	order_items	order_id	orders	order_id
	order_items	product_id	products	product_id
	orders	customer_id	customers	customer_id

KEY_COLUMN_USAGE 29 x

Read Only

Output

insert an order with customer_id = 999: -

358
359 A Foreign Key is used to connect tables and maintain relationships between them.
360
361 Example:
362 INSERT INTO orders
363 VALUES (1011, 999, '2024-09-01', 'Pending', 1000);
364
365 ✖ If we try to insert an order with customer_id = 999, the database will return an error because this customer ID does not exist in the customers:
366 This happens due to the Foreign Key constraint, which prevents invalid data from being inserted and maintains data consistency between related
367

Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 30	13:27:19	SELECT o.order_id, o.order_date, c.first_name, c.last_name, o.total_amount FROM orders o INNER JOIN cust...	13 row(s) returned	0.000 sec / 0.000 sec
✓ 31	13:28:35	SELECT c.customer_id, c.first_name, c.last_name, o.order_id, o.order_date, o.total_amount FROM customers ...	13 row(s) returned	0.000 sec / 0.000 sec
✓ 32	13:29:12	SELECT o.order_id, p.product_name, oi.quantity, oi.unit_price, oi.discount_pct FROM orders o INNER JOIN or...	17 row(s) returned	0.000 sec / 0.000 sec
✓ 33	13:31:00	SELECT c.first_name, o.order_id FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id L...	13 row(s) returned	0.000 sec / 0.000 sec
✓ 34	13:31:32	SELECT c.first_name, o.order_id FROM customers c RIGHT JOIN orders o ON c.customer_id = o.customer_id ...	13 row(s) returned	0.000 sec / 0.000 sec
✓ 35	13:32:33	SELECT TABLE_NAME AS child_table, COLUMN_NAME AS foreign_key_column, REFERENCED_T...	3 row(s) returned	0.000 sec / 0.000 sec
✗ 36	13:32:57	INSERT INTO orders VALUES (1011, 999, '2024-09-01', 'Pending', 1000)	Error Code: 1062. Duplicate entry '1011' for key 'orders.PRIMARY'	0.031 sec

SECTION – E

Q24. Write a query using CASE to classify products into price tiers:

- 'Budget' → unit_price < 1000
- 'Mid-Range' → unit_price BETWEEN 1000 AND 3000
- 'Premium' → unit_price > 3000

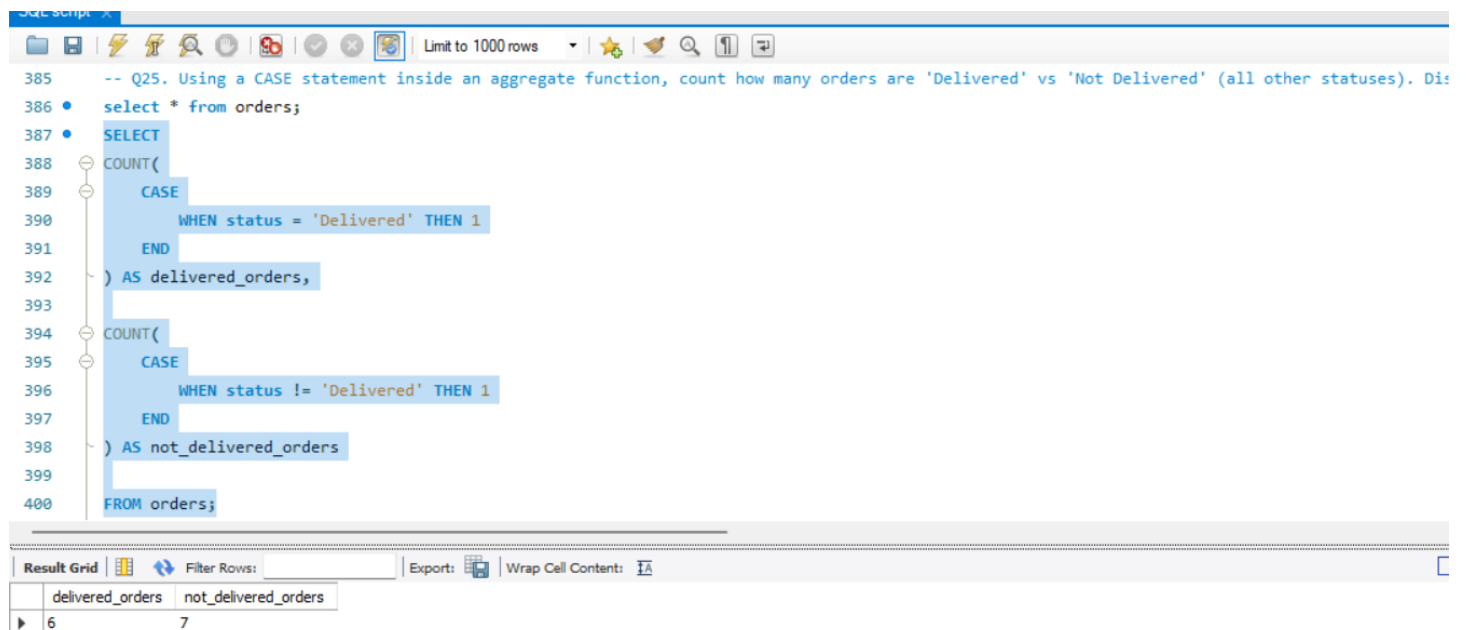
Display: product_name, unit_price, price_tier.

```
367
368  -- Section E – Advanced Concepts (CASE, ACID, Transactions) --
369
370  /* Q24. Write a query using CASE to classify products into price tiers:
371     • 'Budget' → unit_price < 1000
372     • 'Mid-Range' → unit_price BETWEEN 1000 AND 3000
373     • 'Premium' → unit_price > 3000
374     Display: product_name, unit_price, price_tier. */
375  SELECT
376     product_name,
377     unit_price,
378     CASE
379         WHEN unit_price < 1000 THEN 'Budget'
380         WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'
381         ELSE 'Premium'
382     END AS price_tier
383  FROM products;
384
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	product_name	unit_price	price_tier
▶	Wireless Earbuds	1499.00	Mid-Range
	Cotton T-Shirt	799.00	Budget
	Smart Watch	2999.00	Mid-Range
	Running Shoes	4599.00	Premium
	Bluetooth Speaker	3499.00	Premium
	Bedsheet Set	1299.00	Mid-Range
	Laptop Stand	899.00	Budget
	Cushion Covers (Set)	599.00	Budget

Q25. Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered' (all other statuses). Display the result in a single row.



```

385 -- Q25. Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered' (all other statuses). Di
386 • select * from orders;
387 • SELECT
388   COUNT(
389     CASE
390       WHEN status = 'Delivered' THEN 1
391     END
392   ) AS delivered_orders,
393
394   COUNT(
395     CASE
396       WHEN status != 'Delivered' THEN 1
397     END
398   ) AS not_delivered_orders
399
400 FROM orders;

```

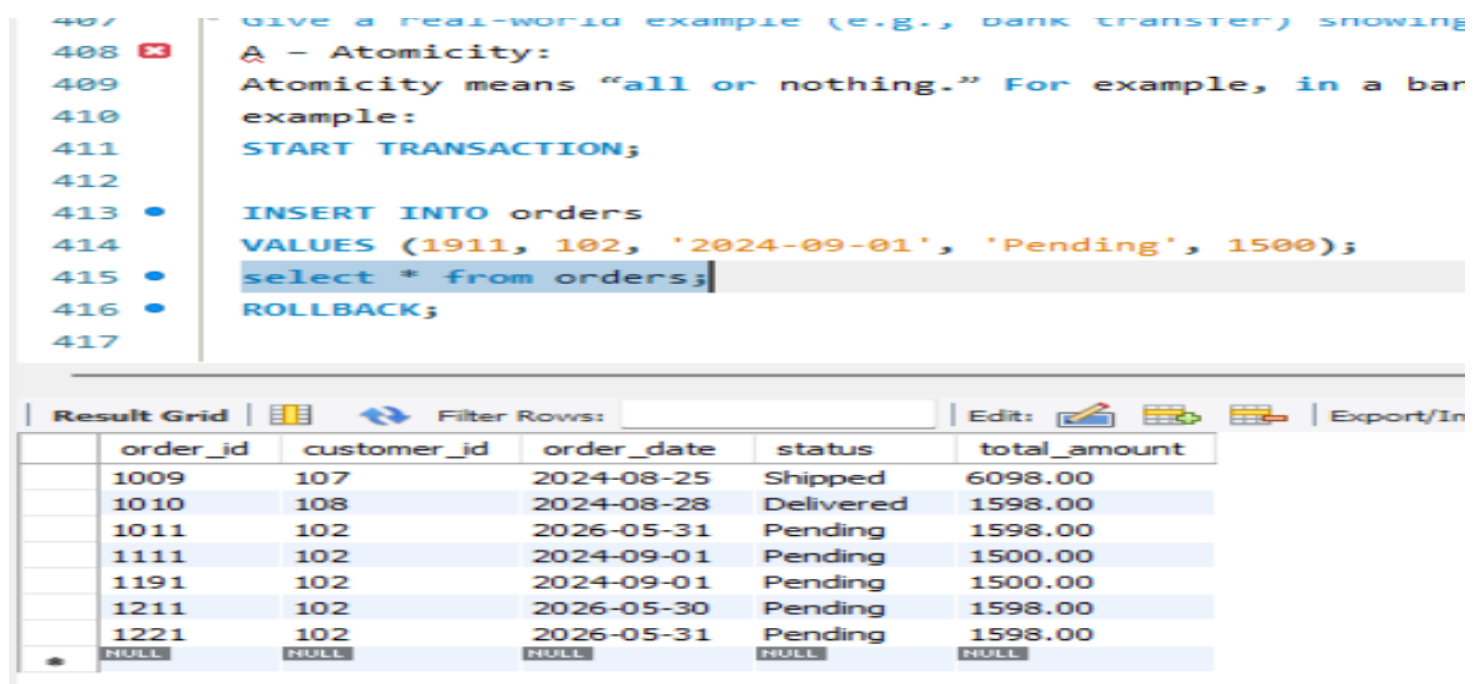
delivered_orders	not_delivered_orders
6	7

Q26. Explain each letter of ACID:

- A – Atomicity
- C – Consistency
- I – Isolation
- D – Durability

Give a real-world example (e.g., bank transfer) showing why each property is important.

A - Atomicity



```

407
408 • Give a real-world example (e.g., bank transfer) showing
409 • A - Atomicity:
410 • Atomicity means "all or nothing." For example, in a bar
411 • example:
412 • START TRANSACTION;
413 •
414 • INSERT INTO orders
415 • VALUES (1911, 102, '2024-09-01', 'Pending', 1500);
416 • select * from orders;
417 • ROLLBACK;

```

order_id	customer_id	order_date	status	total_amount
1009	107	2024-08-25	Shipped	6098.00
1010	108	2024-08-28	Delivered	1598.00
1011	102	2026-05-31	Pending	1598.00
1111	102	2024-09-01	Pending	1500.00
1191	102	2024-09-01	Pending	1500.00
1211	102	2026-05-30	Pending	1598.00
1221	102	2026-05-31	Pending	1598.00
NULL	NULL	NULL	NULL	NULL

C – Consistency

```
417
418 ❌ C – Consistency:
419 Consistency means data remains correct and valid before and after a transaction. For example, account balances should remain accurate.
420 example:
421 INSERT INTO products
422 VALUES
423 (301, 'Laptop', 'Electronics', 'Dell', -50, 10);
424 ❌ error because rule:- CHECK (unit_price > 0)
425
```

Output

Action Output

#	Time	Action	Message
✓ 31	13:28:35	SELECT c.customer_id, c.first_name, c.last_name, o.order_id, o.order_date, o.total_amount FROM customers ...	13 row(s) returned
✓ 32	13:29:12	SELECT o.order_id, p.product_name, oi.quantity, oi.unit_price, oi.discount_pct FROM orders o INNER JOIN or...	17 row(s) returned
✓ 33	13:31:00	SELECT c.first_name, o.order_id FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id L...	13 row(s) returned
✓ 34	13:31:32	SELECT c.first_name, o.order_id FROM customers c RIGHT JOIN orders o ON c.customer_id = o.customer_id ...	13 row(s) returned
✓ 35	13:32:33	SELECT TABLE_NAME AS child_table, COLUMN_NAME AS foreign_key_column, REFERENCED_T...	3 row(s) returned
✗ 36	13:32:57	INSERT INTO orders VALUES (1011, 999, '2024-09-01', 'Pending', 1000)	Error Code: 1062. Duplicate entry '1011' for key 'orders.PRIMARY'
✓ 37	13:34:18	SELECT product_name, unit_price, CASE WHEN unit_price < 1000 THEN 'Budget' WHEN unit_price BETWE...	8 row(s) returned
✓ 38	13:35:05	SELECT COUNT(CASE WHEN status = 'Delivered' THEN 1 END) AS delivered_orders, COUNT(...	1 row(s) returned
✗ 39	18:35:44	INSERT INTO orders VALUES (1011, 102, '2024-09-01', 'Pending', 1500)	Error Code: 1062. Duplicate entry '1011' for key 'orders.PRIMARY'
✗ 40	18:36:07	INSERT INTO orders VALUES (1011, 102, '2024-09-01', 'Pending', 1500)	Error Code: 1062. Duplicate entry '1011' for key 'orders.PRIMARY'
✗ 41	18:37:10	INSERT INTO orders VALUES (1011, 102, '2024-09-01', 'Pending', 1500)	Error Code: 1062. Duplicate entry '1011' for key 'orders.PRIMARY'
✗ 42	18:37:37	INSERT INTO orders VALUES (1011, 102, '2024-09-01', 'Pending', 1500)	Error Code: 1062. Duplicate entry '1011' for key 'orders.PRIMARY'
✓ 43	18:37:46	START TRANSACTION	0 row(s) affected
✗ 44	18:37:58	INSERT INTO orders VALUES (1111, 102, '2024-09-01', 'Pending', 1500)	Error Code: 1062. Duplicate entry '1111' for key 'orders.PRIMARY'
✓ 45	18:38:09	INSERT INTO orders VALUES (1911, 102, '2024-09-01', 'Pending', 1500)	1 row(s) affected
✓ 46	18:38:16	ROLLBACK	0 row(s) affected
✗ 47	18:40:05	INSERT INTO products VALUES (301, 'Laptop', 'Electronics', 'Dell', -50, 10)	Error Code: 3819. Check constraint 'products_chk_1' is violated.

I – Isolation

```
430
431 • UPDATE orders
432 SET total_amount = 2000
433 WHERE order_id = 1001;
434 • select * from orders;
435
436 ❌ D – Durability:
437 Durability means once a transaction is saved, it
438 example:
```

Result Grid					
Filter Rows:					
Edit:   					
	order_id	customer_id	order_date	status	total_amount
▶	1001	101	2024-08-01	Delivered	2000.00
	1002	102	2024-08-03	Delivered	799.00
	1003	103	2024-08-05	Shipped	7498.00
	1004	101	2024-08-10	Delivered	3499.00
	1005	104	2024-08-12	Cancelled	2999.00
	1006	105	2024-08-15	Delivered	5898.00
	1007	106	2024-08-18	Pending	1299.00
	1008	103	2024-08-20	Delivered	899.00
	1009	107	2024-08-25	Shipped	6098.00

orders 41 x

D – Durability

```
436 Q - Durability:
437 Durability means once a transaction is saved, it stays saved permanently even if the system crashes or power fails.
438 example:
439 START TRANSACTION;
440
441 INSERT INTO orders
442 VALUES
443 (1171, 102, '2024-09-01', 'Pending', 1500);
444 select * from orders;
445 COMMIT;
```

Result Grid

order_id	customer_id	order_date	status	total_amount
1171	102	2024-09-01	Pending	1500.00
1191	102	2024-09-01	Pending	1500.00
1211	102	2026-05-30	Pending	1598.00
1221	102	2026-05-31	Pending	1598.00
NULL	NULL	NULL	NULL	NULL

Q27. Write a SQL transaction that does the following atomically:

1. Insert a new order (order_id=1011, customer_id=102, today's date, 'Pending', 1598.00)
2. Insert two order items for that order
3. Update the stock_qty of the purchased products
4. If any step fails, ROLLBACK the entire transaction. Otherwise, COMMIT.

Write the complete BEGIN...COMMIT/ROLLBACK block.

1. Insert a new order (order_id=1011, customer_id=102, today's date, 'Pending', 1598.00)

```
453 START TRANSACTION;
454
455 -- Insert new order --
456 INSERT INTO orders
457 VALUES
458 (1011, 102, CURDATE(), 'Pending', 1598.00);
459 select * from orders;
```

Result Grid

order_id	customer_id	order_date	status	total_amount
1011	102	2026-05-31	Pending	1598.00
1111	102	2024-09-01	Pending	1500.00
1191	102	2024-09-01	Pending	1500.00
1211	102	2026-05-30	Pending	1598.00
1221	102	2026-05-31	Pending	1598.00

orders 32

Output

#	Time	Action	Message
49	18:42:27	INSERT INTO orders VALUES (1111, 102, '2024-09-01', 'Pending', 1500)	Error Code: 1062. Duplicate entry '1111' for key 'orders.PRIMARY'
50	18:42:36	INSERT INTO orders VALUES (1191, 102, '2024-09-01', 'Pending', 1500)	1 row(s) affected
51	18:43:09	COMMIT	0 row(s) affected
52	18:44:08	INSERT INTO orders VALUES (1211, 102, CURDATE(), 'Pending', 1598.00)	Error Code: 1062. Duplicate entry '1211' for key 'orders.PRIMARY'
53	18:44:10	INSERT INTO orders VALUES (1211, 102, CURDATE(), 'Pending', 1598.00)	Error Code: 1062. Duplicate entry '1211' for key 'orders.PRIMARY'
54	18:44:25	INSERT INTO orders VALUES (1221, 102, CURDATE(), 'Pending', 1598.00)	1 row(s) affected
55	18:46:00	INSERT INTO orders VALUES (1011, 102, CURDATE(), 'Pending', 1598.00)	Error Code: 1062. Duplicate entry '1011' for key 'orders.PRIMARY'
56	18:49:34	DELETE FROM orders WHERE order_id = 1011	Error Code: 1451. Cannot delete or update a parent row: a foreign key constraint fails
57	18:50:21	DELETE FROM order_items WHERE order_id = 1011	2 row(s) affected
58	18:50:26	DELETE FROM orders WHERE order_id = 1011	1 row(s) affected
59	18:50:34	INSERT INTO orders VALUES (1011, 102, CURDATE(), 'Pending', 1598.00)	1 row(s) affected
60	18:51:01	select * from orders LIMIT 0, 1000	15 row(s) returned

2. Insert two order items for that order :-

For first order:

```
462 -- Insert first order item --
463 • INSERT INTO order_items
464 VALUES
465 (5016, 1011, 206, 1, 1299.00, 0);
466 • select * from order_items;
467
468 -- Insert second order item --
469 • INSERT INTO order_items
470 VALUES
471 (5017, 1011, 208, 1, 299.00, 0);
```

Result Grid

item_id	order_id	product_id	quantity	unit_price	discount_pct
5011	1008	207	1	899.00	0.00
5012	1009	205	1	3499.00	0.00
5013	1009	208	2	599.00	15.00
5014	1010	206	1	1299.00	0.00
5015	1010	208	1	599.00	0.00
5016	1011	206	1	1299.00	0.00
•	NULL	NULL	NULL	NULL	NULL

order_items 33 x

Output

Action Output

#	Time	Action	Message
✓ 57	18:50:21	DELETE FROM order_items WHERE order_id = 1011	2 row(s) affected
✓ 58	18:50:26	DELETE FROM orders WHERE order_id = 1011	1 row(s) affected
✓ 59	18:50:34	INSERT INTO orders VALUES (1011, 102, CURDATE(), 'Pending', 1598.00)	1 row(s) affected
✓ 60	18:51:01	select * from orders LIMIT 0, 1000	15 row(s) returned
✓ 61	18:52:28	INSERT INTO order_items VALUES (5016, 1011, 206, 1, 1299.00, 0)	1 row(s) affected
✓ 62	18:53:47	select * from order_items LIMIT 0, 1000	16 row(s) returned

For second order:

```
467
468 -- Insert second order item --
469 • INSERT INTO order_items
470 VALUES
471 (5017, 1011, 208, 1, 299.00, 0);
472 • select * from order_items;
473 -- Update stock quantity --
474 • UPDATE products
```

Result Grid

item_id	order_id	product_id	quantity	unit_price	discount_pct
5012	1009	205	1	3499.00	0.00
5013	1009	208	2	599.00	15.00
5014	1010	206	1	1299.00	0.00
5015	1010	208	1	599.00	0.00
5016	1011	206	1	1299.00	0.00
5017	1011	208	1	299.00	0.00
•	NULL	NULL	NULL	NULL	NULL

3. Update the stock_qty of the purchased products

```
473
474 -- 3. Update the stock_qty of the purchased products --
475 • UPDATE products
476 SET stock_qty = stock_qty - 1
477 WHERE product_id = 206;
478
479 • UPDATE products
480 SET stock_qty = stock_qty - 1
481 WHERE product_id = 208;
```

Output				
Action Output				
#	Time	Action	Message	
✗ 56	18:49:34	DELETE FROM orders WHERE order_id = 1011	Error Code: 1451. Cannot delete or update a parent row: a foreign key constraint fails (t	
✓ 57	18:50:21	DELETE FROM order_items WHERE order_id = 1011	2 row(s) affected	
✓ 58	18:50:26	DELETE FROM orders WHERE order_id = 1011	1 row(s) affected	
✓ 59	18:50:34	INSERT INTO orders VALUES (1011, 102, CURDATE(), 'Pending', 1598.00)	1 row(s) affected	
✓ 60	18:51:01	select * from orders LIMIT 0, 1000	15 row(s) returned	
✓ 61	18:52:28	INSERT INTO order_items VALUES (5016, 1011, 206, 1, 1299.00, 0)	1 row(s) affected	
✓ 62	18:53:47	select * from order_items LIMIT 0, 1000	16 row(s) returned	
✓ 63	18:55:15	INSERT INTO order_items VALUES (5017, 1011, 208, 1, 299.00, 0)	1 row(s) affected	
✓ 64	18:55:25	select * from order_items LIMIT 0, 1000	17 row(s) returned	
✓ 65	18:56:15	UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 206	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	
✗ 66	18:57:44	select product_id, stock_qty from products where product_qty=2006 LIMIT 0, 1000	Error Code: 1054. Unknown column 'product_qty' in 'where clause'	
✓ 67	18:58:11	select product_id, stock_qty from products where product_id=2006 LIMIT 0, 1000	0 row(s) returned	
✗ 68	18:58:20	Create database Celebal	Error Code: 1007. Can't create database 'celebal'; database exists	
✓ 69	18:58:41	UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 208	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	
✓ 70	18:58:48	UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 206	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	

4. If any step fails, ROLLBACK the entire transaction. Otherwise, COMMIT.

```
482
483 -- 4. If any step fails, ROLLBACK the entire transaction. Otherwise, COMMIT. --
484 -- Save changes --
485 • COMMIT;
486
487 -- If any step fails --
488 • ROLLBACK;
489
490
491
492
```

Output				
Action Output				
#	Time	Action	Message	
✓ 65	18:56:15	UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 206	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	
✗ 66	18:57:44	select product_id, stock_qty from products where product_qty=2006 LIMIT 0, 1000	Error Code: 1054. Unknown column 'product_qty' in 'where clause'	
✓ 67	18:58:11	select product_id, stock_qty from products where product_id=2006 LIMIT 0, 1000	0 row(s) returned	
✗ 68	18:58:20	Create database Celebal	Error Code: 1007. Can't create database 'celebal'; database exists	
✓ 69	18:58:41	UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 208	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	
✓ 70	18:58:48	UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 206	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	
✓ 71	18:59:25	COMMIT	0 row(s) affected	
✓ 72	18:59:29	ROLLBACK	0 row(s) affected	